

The EYES – Requirements Notes

Date: 2026-05-10

This draft keeps only the three requested requirement areas and expands them into a form suitable for a client-facing PDF. The intent is to define the rules clearly enough that the team can build, audit, and validate the behavior without ambiguity.

Document purpose

- Capture the client's three required areas in plain language.
- Turn each area into explicit rules, expected behavior, and open questions.
- Provide enough content to serve as a short requirements PDF for review and sign-off.

Reading note

- Each section below is written as a requirement set, not a technical design.
- The client can approve, reject, or refine the rules before implementation.

1. Privacy Shields

Requirement

- Entries will never be indexed when the privacy shield is enabled.
- The system must clearly separate content that can be stored, searched, audited, or exported from content that is protected by the privacy shield.
- The privacy decision must be visible to users before sync or ingestion starts.

Business goal

- Users should have a reliable way to mark content as private so it never appears in memory search or audit surfaces.
- The product should avoid accidentally exposing sensitive entries in summaries, embeddings, or connector views.

What this means in practice

- Protected entries should be excluded from semantic search, keyword search, embeddings, audit summaries, and cross-connector memory views.
- If an entry is protected, it should not silently flow into downstream processing.
- Users should know whether the entry was blocked, stored privately, or ignored entirely.
- The system should treat privacy as an explicit control, not as a hidden backend behavior.
- If a protected entry is synced again later, the same privacy decision should still apply unless the user changes it.

Rules to define

- Which sources are covered by the privacy shield.
- Whether the rule applies to all connectors or only selected data types.
- Whether exceptions are allowed for admin-only troubleshooting.
- Whether protected entries can be restored later or remain permanently excluded.
- Whether the shield is enabled at the workspace level, connector level, or item level.
- Whether the shield applies to historical records already present in the system.
- Whether the shield should remove previously indexed items or only block future indexing.

Suggested rule categories

- Source rule: one connector may be blocked while another remains allowed.
- Content rule: certain fields may be allowed while the full body is blocked.
- Time rule: content may be stored briefly for processing and then removed.
- Role rule: admins may see status metadata while users only see the result.

Acceptance criteria

- When a privacy shield is active, protected entries do not appear in search results.
- Protected entries do not generate embeddings for retrieval use.
- The UI shows a clear protected state rather than silently failing.
- The system records that the entry was shielded without revealing protected content.

Questions to confirm with the client

- Should protected entries be completely ignored, or stored without indexing?
- Should the privacy shield apply to all future syncs automatically?

- Should the UI show a badge, status, or audit trail when an entry is shielded?
- Should users be able to reverse the shield later for previously protected items?
- Should the shield behavior be consistent across all connectors and data types?
- Should the client approve a list of connectors that are always considered sensitive?

Open concerns

- If the system stores raw data but does not index it, the team still needs a clear policy for retention and deletion.
- If the system supports export, export rules must match the privacy shield rules so protected content is not accidentally included.
- If protected entries are used for debugging, that access must be logged and limited.

2. Audit Rules

Requirement

- Audit behavior must be based on the privacy and indexing rules above.
- The audit should explain why a record was indexed, skipped, hidden, or protected.
- The system must produce a clear audit result instead of only a raw model output.

Business goal

- Audits should make the system explainable to both users and administrators.
- Any decision that changes data visibility or indexing should leave a trace that can be reviewed later.

What the audit should answer

- Was the entry indexed or not?
- If not indexed, which rule caused the decision?
- Was the audit generated from a rule, a connector event, or a manual review?
- Is the action reversible, and if so, by whom?
- Was the decision automatic, manual, or a fallback result?
- Was the rule applied because of privacy, validation, sync failure, or a system exception?
- Can the audit be filtered by connector, user, date, or rule type?

Suggested audit rule format

- Rule name
- Rule condition
- Decision
- Reason
- Scope
- Effective date
- Owner
- Priority
- Review status

Example audit categories

- Privacy audit: content was excluded because the privacy shield was enabled.
- Sync audit: content was skipped because the connector failed or the payload was invalid.
- Content audit: content was blocked because it did not meet formatting or safety rules.
- Manual audit: content was reviewed and overridden by an authorized user.

Example

- Rule name: Privacy Shield
- Rule condition: If the user marks an entry as private
- Decision: Do not index
- Reason: Protected by user privacy setting
- Scope: All search and memory surfaces
- Effective date: Immediate
- Owner: Product or compliance team
- Review status: Active

Audit outputs to include

- Entry status: indexed / skipped / protected / pending
- Reason code: privacy, sync failure, invalid content, manual override
- Connector name
- Timestamp

- Optional reviewer note
- Audit ID
- Source item ID
- Rule ID or policy reference
- Whether a retry is allowed

Acceptance criteria

- Every important rule decision can be explained in one audit record.
- Audit records are consistent and searchable.
- Users and admins can understand why a record changed state without reading raw logs.
- The audit output is stable enough to support support tickets and compliance review.

Questions to confirm with the client

- Should audit results be visible to the user or only to admins?
- Should every skipped entry create an audit log record?
- Should audits be immutable once written?
- What should happen when a rule conflicts with another rule?
- Should audit notes be editable after the fact?
- Should a manual review always override an automatic rule?
- Should the audit display the exact rule text or only a summary?

Open concerns

- A noisy audit trail can become hard to read unless records are grouped and filtered well.
- If audits are generated from model output, the model format must be controlled carefully.
- If audit rules can conflict, the resolution order must be defined before implementation.

3. Keys & Access

Requirement

- Claude AI fresh key needed.
- The system should support key rotation without breaking existing users.
- The app should fail clearly when a key is missing, invalid, or expired.

Business goal

- The app should remain usable when keys are changed, rotated, or revoked.
- Key handling should be explicit so developers and operators know what is required.

What this means in practice

- The Claude key should be stored securely and treated as a required secret.
- The system should distinguish between a missing key, an invalid key, and a rate-limited key.
- If fallback providers exist, the app should define when fallback is allowed and when it is not.
- Key failures should be reported clearly enough that the team can diagnose setup problems quickly.
- The system should not silently use the wrong key or fall back in a way that confuses the user.

Rules to define

- Who provides the fresh key.
- How often the key can or must be rotated.
- Whether the key is per environment, per tenant, or shared across the app.
- Whether fallback to another provider is allowed if Claude fails.
- Whether old keys are revoked immediately or kept for a short transition period.
- Whether the same key can be reused in development and production.
- Whether key status should be checked at startup, on demand, or both.

Suggested key handling rules

- Required secret rule: the app cannot start without the required key in the target environment.
- Rotation rule: a new key must be added before the old key expires.
- Validation rule: invalid or expired keys should produce a clear error state.
- Fallback rule: fallback providers may be used only if the client approves that behavior.

Acceptance criteria

- A missing Claude key is reported clearly.
- An invalid or expired key does not fail silently.
- The team can rotate keys without changing unrelated app behavior.
- The application makes it obvious whether it is using Claude or another provider.

Questions to confirm with the client

- Should we request one fresh key for development and another for production?
- Should expired keys block the feature entirely or degrade gracefully?
- Should key rotation be manual or supported by an automated secret manager?
- Should the UI display a connector error when the key is invalid?
- Should fallback to another provider be allowed when Claude is unavailable?
- Should there be different keys per environment or one shared project key?
- Should key usage be logged for security review?

Open concerns

- Key rotation without a clear process can lead to outages.
- Shared keys across environments can make debugging and security review harder.
- If fallback behavior is allowed, users need to know when the system is no longer using the primary model.

Recommended final structure for the PDF

- Privacy Shields
- Audit Rules
- Keys & Access
- Open Questions
- Approval Notes

If you want, I can also turn this into a more formal requirements document with headings like Scope, Rules, Audit, Security, and Open Questions.